TRƯỜNG ĐẠI HỌC SÀI GÒN (SGU) • KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO PHẦN 2: PHÂN CẤP CON NGƯỜI & ĐA HÌNH CHIẾT KHẤU

ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THỜI TRANG (JAVA OOP) • PHẦN 2

Phân công: Thành viên 2 | GVHD: Thầy Trần Ngọc Minh Hiếu (tnmhieu@sgu.edu.vn)

HƯỚNG DẪN HỌC & BẢO VỆ VẤN ĐÁP ĐỒ ÁN OOP (ĐIỂM 10/10)

> **Phân công thành viên:** Thành viên 2

> **Các lớp phụ trách (8 lớp & interface):** `Nguoi` , `KhachHang` , `KhachHangVIP` , `KhachHangThanThiet` , `NhanVien` , `NhanVienQuanLy` , `NhaCungCap` , `IGiamGiaKhachHang` .

> **Nhiệm vụ cốt lõi:** Nắm vững cấu trúc phân cấp kế thừa con người, cơ chế chiết khấu đa hình thông qua interface `IGiamGiaKhachHang` , tính năng quản lý đối tác nhà cung cấp và bảo vệ dữ liệu người dùng.

1. CÂY PHÂN CẤP KẾ THỪA CON NGƯỜI (HUMAN HIERARCHY)



- **Lớp gốc `Nguoi`**: Nắm giữ các thuộc tính định danh cá nhân căn bản nhất.
- **Nhánh `KhachHang`**: Quản lý các phân khúc khách hàng mua sắm tại cửa hàng.
- **Nhánh `NhanVien`**: Quản lý nhân sự làm việc tại cửa hàng, hưởng chính sách mua hàng nội bộ.
- **Đa hình chiết khấu qua Interface `IGiamGiaKhachHang`**: Thống nhất phương thức `applyDiscount(double)` cho các đối tượng được hưởng ưu đãi giảm giá.

2. GIẢI THÍCH CHI TIẾT TỪNG ĐOẠN CODE CỐT LÕI

Đoạn 1: Lớp cơ sở `Nguoi` & Tính Đóng gói (Encapsulation)

```
public class Nguoi {
    private String name;
    private String numberPhone;

    public Nguoi() {
        this.name = "";
        this.numberPhone = "";
    }
```

```
    public Nguoi(String name, String numberPhone) {
        this.name = (name != null) ? name.trim().replace(",", " ") : "";
        this.numberPhone = (numberPhone != null) ? numberPhone.trim().replace(",", "") : "";
    }

    public String getName() { return name; }
    public void setName(String name) {
        if (name != null && !name.trim().isEmpty()) {
            this.name = name.trim().replace(",", " ");
        }
    }

    public String getNumberPhone() { return numberPhone; }
    public void setNumberPhone(String numberPhone) {
        if (numberPhone != null) {
            this.numberPhone = numberPhone.trim().replace(",", "");
        }
    }
}
```

- **Giải thích từng dòng code:**
- `private String name; private String numberPhone;` : Thuộc tính được bảo vệ chặt chẽ, không cho phép truy cập trực tiếp từ bên ngoài lớp.
- `name.trim().replace(",", " ")` : Cắt bỏ khoảng trắng thừa ở 2 đầu chuỗi và thay thế dấu phẩy bằng khoảng trắng. Giữ an toàn cấu trúc khi lưu xuống file `users.txt` (định dạng CSV).
- `numberPhone.trim().replace(",", "")` : Chuẩn hóa số điện thoại, loại bỏ các ký tự dấu phẩy vô tình gõ nhầm.
- Các hàm `setName` và `setNumberPhone` kiểm tra dữ liệu đầu vào không được null hoặc rỗng trước khi cập nhật.

Đoạn 2: Kế thừa trung gian `KhachHang`

```
public class KhachHang extends Nguoi {
    private String LoaiKH;

    public KhachHang() {
        super("", "");
        this.LoaiKH = "Thuong";
    }

    public KhachHang(String name, String numberPhone, String LoaiKH) {
        super(name, numberPhone);
        this.LoaiKH = (LoaiKH != null) ? LoaiKH : "Thuong";
    }

    public String getLoaiKH() { return LoaiKH; }
    public void setLoaiKH(String LoaiKH) { this.LoaiKH = LoaiKH; }
}
```

- **Giải thích từng dòng code:**
- `public class KhachHang extends Nguoi` : Quan hệ kế thừa. `KhachHang` kế thừa họ tên và số điện thoại từ `Nguoi` .

- `super(name, numberPhone)` : Gọi constructor của lớp cha `Nguoi` để gán giá trị cho `name` và `numberPhone` .
- `private String LoaiKH` : Thuộc tính phân loại khách hàng (Thường, VIP, Thân thiết). Mặc định nếu không truyền là `"Thuong"` .

Đoạn 3: Interface `IGiamGiaKhachHang`

```
public interface IGiamGiaKhachHang {
    double applyDiscount(double totalPrice);
}
```

- **Giải thích từng dòng code:**
- Định nghĩa một hợp đồng giao tiếp (Contract). Bất kỳ đối tượng nào tham gia vào hệ thống thanh toán có chính sách ưu đãi giảm giá đều phải hiện thực hàm `applyDiscount` .
- Đầu vào là `totalPrice` (tổng tiền trước giảm), đầu ra là tổng tiền sau khi đã trừ đi số tiền được chiết khấu.

Đoạn 4: Khách hàng VIP `KhachHangVIP` giảm 20%

```
public class KhachHangVIP extends KhachHang implements IGiamGiaKhachHang {
    private String cardCodeVIP;

    public KhachHangVIP(String name, String numberPhone, String cardCodeVIP) {
        super(name, numberPhone, "VIP");
        this.cardCodeVIP = (cardCodeVIP != null) ? cardCodeVIP.trim().replace(",", " ") : "";
    }

    public String getCardCodeVIP() { return cardCodeVIP; }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.2);
    }
}
```

- **Giải thích từng dòng code:**
- `extends KhachHang implements IGiamGiaKhachHang` : Kế thừa thuộc tính khách hàng và cam kết cài đặt chiết khấu.
- `private String cardCodeVIP` : Thuộc tính riêng của khách VIP là mã số thẻ định danh VIP.
- `@Override public double applyDiscount(...)` : Ghi đè phương thức từ interface. Công thức: `totalPrice - (totalPrice * 0.2)` -> Giảm trực tiếp **20%** tổng giá trị hóa đơn.

Đoạn 5: Khách hàng Thân thiết `KhachHangThanThiet` giảm 10% & Tích điểm

```
public class KhachHangThanThiet extends KhachHang implements IGiamGiaKhachHang {
    private int diemTichLuy;

    public KhachHangThanThiet(String name, String numberPhone, int diemTichLuy) {
        super(name, numberPhone, "ThanThiet");
        this.diemTichLuy = (diemTichLuy >= 0) ? diemTichLuy : 0;
    }

    public int getDiemTichLuy() { return diemTichLuy; }
```

```
    public void setDiemTichLuy(int diemTichLuy) {
        if (diemTichLuy >= 0) this.diemTichLuy = diemTichLuy;
    }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.10);
    }
}
```

- **Giải thích từng dòng code:**
- `private int diemTichLuy` : Quản lý điểm thưởng tích lũy của khách hàng thân thiết. Ràng buộc điểm không được âm (`diemTichLuy >= 0`).
- `@Override public double applyDiscount(...)` : Chiết khấu ưu đãi **10%** tổng hóa đơn cho khách thân thiết.

Đoạn 6: Nhánh Nhân viên `NhanVien` & `NhanVienQuanLy`

```
public class NhanVien extends Nguoi implements IGiamGiaKhachHang {
    private String staffID;

    public NhanVien(String name, String numberPhone, String staffID) {
        super(name, numberPhone);
        this.staffID = (staffID != null) ? staffID.trim().replace(",", " ") : "";
    }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.5);
    }
}
```

```
public class NhanVienQuanLy extends NhanVien {
    private double phuCapTrachNhiem;

    public NhanVienQuanLy(String name, String numberPhone, String staffID, double phuCapTrachNhiem)
{
        super(name, numberPhone, staffID);
        this.phuCapTrachNhiem = (phuCapTrachNhiem >= 0) ? phuCapTrachNhiem : 0;
    }

    public double getPhuCapTrachNhiem() { return phuCapTrachNhiem; }
}
```

- **Giải thích từng dòng code:**
- `NhanVien` kế thừa từ `Nguoi` và cài đặt `IGiamGiaKhachHang` .
- Mức giảm giá nội bộ dành cho nhân viên mua sắm là **50%** (`totalPrice * 0.5`).
- `NhanVienQuanLy` kế thừa từ `NhanVien` : Quản lý cũng là một nhân viên, do đó tự động được hưởng mức giảm giá 50% từ lớp cha.
- `private double phuCapTrachNhiem` : Thuộc tính riêng mở rộng của Quản lý, quản lý khoản tiền phụ cấp chức vụ hàng tháng.

Đoạn 7: Lớp độc lập `NhaCungCap`

```
public class NhaCungCap {
    private String maNCC;
    private String tenNCC;
    private String diaChi;
    private String soDienThoai;
```

- **Giải thích:** Quản lý danh mục đối tác may mặc cung ứng hàng hóa cho shop thời trang. Đóng gói đầy đủ getter/setter và phương thức in thông tin chuẩn hàng ngang (`hienThiThongTin()`), phục vụ trực tiếp cho Menu 9 trong hệ thống.

3. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU & CÁCH TRẢ LỜI ĐẠT 10 ĐIỂM

CÂU HỎI VẤN ĐÁP: Tại sao cả `KhachHangVIP` và `NhanVien` đều cài đặt interface `IGiamGiaKhachHang`? Tại sao không viết phương thức này trong lớp `Nguoi`?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, nếu đặt phương thức giảm giá ở lớp `Nguoi`, thì mọi người bình thường bước vào shop đều có quyền giảm giá, điều này trái với thực tế kinh doanh. Khách hàng vãng lai thông thường không được giảm giá. Vì vậy, nhóm sử dụng Interface `IGiamGiaKhachHang` để đại diện cho một "năng lực đặc biệt" (Capability). Chỉ những đối tượng thuộc diện ưu đãi như VIP (20%), Thân thiết (10%) hoặc Nhân viên nội bộ (50%) mới được gắn interface này.

CÂU HỎI VẤN ĐÁP: Trong lớp `HoaDon`, khi tính tổng tiền, cơ chế đa hình chiết khấu diễn ra như thế nào?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, trong `HoaDon`, nhóm lưu trữ một thuộc tính kiểu cha: `private Nguoi person`. Khi tính tiền:
```java
if (person instanceof IGiamGiaKhachHang) {
total = ((IGiamGiaKhachHang) person).applyDiscount(total);
}
```
Tại thời điểm chạy (Runtime), máy ảo Java sẽ kiểm tra xem đối tượng thực tế mà con trỏ `person` đang trỏ tới là gì. Nếu là `KhachHangVIP`, nó tự động kích hoạt hàm giảm 20%; nếu là `NhanVien`, nó giảm 50%. Nhờ đa hình, code hoàn toàn không cần các khối lệnh `if-else` kiểm tra từng loại lớp, giúp code ngắn gọn và dễ mở rộng.

CÂU HỎI VẤN ĐÁP: Tại sao `NhanVienQuanLy` không cần viết lại phương thức `applyDiscount` mà vẫn được giảm giá 50%?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, đó là nhờ **Tính kế thừa (Inheritance)**. `NhanVienQuanLy` kế thừa từ `NhanVien`, mà `NhanVien` đã hiện thực `IGiamGiaKhachHang` và định nghĩa `applyDiscount` giảm 50%. Do lớp Quản lý không ghi đè lại phương thức này, Java sẽ tự động sử dụng phương thức được thừa hưởng từ lớp cha.